

Cell 16 :-

7. DOC_CONTENT_TMPL

This query:

```
</> Python
DOC_CONTENT_TMPL = PREFIXES + """
SELECT ?doc ?label ?content
WHERE {
  VALUES ?doc { %DOCS% }
  ?doc ex:documentContent ?content .
  OPTIONAL { ?doc rdfs:label ?label }
}
"""
```

is not actually used for traversal.

It is used later by Cell 17.

Its purpose is:

```
Retrieved KG node
↓
Find its document content
↓
Use document text as context
```

So:

```
KG relationship
+
Document content
↓
LLM context
```

9. `_tokens()`

`</>` Python

```
def _tokens(s):  
    return {  
        t for t in re.findall(...) if t not in _STOP and len(t) > 2  
    }
```

This converts text into meaningful tokens.

For example:

```
"How do I open the OpenTCCM model using MTCCM?"
```

becomes approximately:

```
open  
opentccm  
model  
mtccm
```

while removing common stop words such as:

```
the  
how  
do  
I  
using
```

This is used by the lexical fallback.

10. `_values_block()`

```
</> Python   Run  
  
def _values_block(uris):  
    return " ".join(f"<{u}>" for u in uris ...)
```

This converts Python RDF node URIs into a SPARQL `VALUES` block.

For example:

```
</> Python   Run  
  
[  
    "http://knwbt.local/kg/OpenTCCM",  
    "http://knwbt.local/kg/MTCCM"  
]
```

becomes:

```
</> sparql   
  
<http://knwbt.local/kg/OpenTCCM>  
<http://knwbt.local/kg/MTCCM>
```

This is how Python passes the seed nodes into the SPARQL query.

11. `link_entities()`

This is the first major function.

```
</> Python   Run  
  
def link_entities(phrases):
```

Input:

```
  
[  
    ("OpenTCCM", "Concept"),  
    ("MTCCM", "System")  
]
```

Output:

```
  
[  
    (KG_node, label, score, phrase),  
    ...  
]
```

In simple terms:

Convert natural-language entity names into actual RDF KG nodes.

If someone asks **“What retriever are you using?”**, the most accurate answer is:

“We are using a Knowledge-Graph-based multi-hop subgraph retriever using SPARQL. It performs entity-to-node linking and then iteratively traverses the RDF Knowledge Graph up to a maximum of 2 hops to retrieve the relevant subgraph.”

Short name

You can call it:

KG-based Multi-Hop SPARQL Retriever

or more specifically:

RDF/SPARQL Multi-Hop Subgraph Retriever

Your retriever consists of

1. **Entity linking** — query entities → KG nodes.
2. **SPARQL retrieval** — queries the RDF Knowledge Graph.
3. **Multi-hop traversal** — expands the graph iteratively.
4. **Maximum hop = 2** — limits traversal depth.
5. **Subgraph retrieval** — collects the relevant relationships around the seed nodes.
6. **No vector retrieval** — no FAISS/embeddings.
7. **No hybrid retrieval** — retrieval is entirely from the KG.

So if your manager asks:

“What retriever are you using in your GraphRAG?”

Say:

“I’m using an RDF Knowledge-Graph-based multi-hop subgraph retriever. Query entities are linked to KG nodes, and SPARQL is used to iteratively traverse the graph up to 2 hops and retrieve the relevant subgraph.”

That's the cleanest and technically accurate description of your current implementation.

how to explain this code in the sentence can give me the point wise what the code is doing in the point wise of the cell 16

Cell 16 — Point-wise Explanation

You can explain **Cell 16** in a meeting in these points:

1. **Cell 16 is the actual KG retrieval stage.**
It takes the entities identified in Cell 15 and retrieves their connected information from the RDF Knowledge Graph.

2. **Entity → KG node linking.**

link_entities() matches each query entity with the corresponding rdfs:label in the Knowledge Graph.

3. **Two matching methods are used.**

- SPARQL substring matching for direct/precise matches.
- RapidFuzz fuzzy matching for approximate matches.

4. **Matched nodes become seed nodes.**

These seed nodes are the starting points for graph traversal.

5. **SPARQL is used for graph retrieval.**

NEIGHBOUR_QUERY_TMPL uses a SPARQL SELECT query to retrieve:

- source node
- predicate/relationship
- target node
- relationship description
- relationship strength
- link type

6. **The traversal checks both directions.**

```
FILTER(?src = ?seed || ?tgt = ?seed)
```

This retrieves relationships where the seed is either the source or the target.

7. **Multi-hop traversal is performed.**

The traverse() function iteratively queries the KG instead of retrieving only direct neighbors.

8. **Maximum traversal depth is controlled by MAX_HOPS.**

In your current implementation:

```
MAX_HOPS = 2
```

So the retriever searches up to **2 hops** from the seed nodes.

9. **The first hop retrieves direct neighbors.**

Seed node → directly connected nodes

10. **The second hop expands the retrieved nodes.**

Seed → Hop 1 → Hop 2

11. The retrieved relationships are stored temporarily in edges.

Each edge contains source, target, predicate, description, strength, link type, hop, and seed score.

12. Duplicate relationships are removed.

The seen set prevents the same (source, predicate, target) relationship from being added multiple times.

13. Traversal is controlled to avoid excessive expansion.

- MAX_FRONTIER = 60 limits nodes expanded at each hop.
- MAX_EDGES = 400 limits the number of retrieved relationships.

14. The result is the relevant KG subgraph.

The retrieved edges and visited nodes are passed to **Cell 17** for ranking and context construction.

Overall flow

Cell 15

Question



Entity phrases



Cell 16

Entity → KG node



Seed nodes



SPARQL SELECT



Hop 1



Hop 2



Retrieved relationships



Relevant KG subgraph



Cell 17

Ranking + context assembly

One sentence for your meeting

“Cell 16 performs KG-based multi-hop subgraph retrieval by linking query entities to RDF nodes and using iterative SPARQL queries to traverse the Knowledge Graph up to two hops, retrieving the relevant relationships and their metadata for the next stage.”

so how can i explain the meet

For the meeting, don't explain every line of code. Explain **the purpose → logic → flow → output**.

You can say this:

“In Cell 16, I implemented the actual Knowledge Graph retrieval. First, the entities extracted from the user query are linked to the corresponding nodes in our RDF Knowledge Graph. For entity linking, I use SPARQL substring matching and fuzzy matching to improve recall.

Once the seed nodes are identified, I use SPARQL SELECT queries to find the relationships connected to those nodes. The query retrieves the source, predicate, target, relationship description, strength, and link type from the RDF graph.

Then I perform multi-hop graph traversal. Currently, the maximum hop is set to two, so the retriever gets the direct neighbors in the first hop and expands to their neighbors in the second hop. The retrieved relationships form the relevant subgraph for the query.

Finally, I pass this retrieved subgraph to Cell 17, where I rank the relationships and assemble the context for answering the question.”

If they ask: “What retriever are you using?”

Say:

“I am using an RDF Knowledge-Graph-based multi-hop subgraph retriever using SPARQL, with a maximum traversal depth of two hops.”

If they ask: “Are you using vector search?”

Say:

“No. For this retriever, I am not using embeddings, FAISS, or a hybrid approach. The retrieval is performed directly from the RDF Knowledge Graph using SPARQL and graph traversal.”

If they ask: “What is the output of Cell 16?”

Say:

“The output is the set of relevant graph relationships and visited nodes, which together represent the query-specific subgraph. This is then used by Cell 17 for ranking and context construction.”

Very short version

If you only have 30 seconds:

“Cell 16 is our actual KG retriever. It takes the query entities, maps them to RDF KG nodes, and uses SPARQL to retrieve their connected relationships. We then perform iterative multi-hop traversal up to two hops to obtain the relevant subgraph. The retrieved source, predicate, target, description, and relationship strength are passed to Cell 17 for ranking and context generation.”

Cell 16 — What I did

- 1. “Cell 16 is the retrieval part of the Graph RAG.”**
- 2. “First, we take the entities identified from the user query and map them to nodes in our Knowledge Graph.”**
- 3. “These matched nodes become our seed nodes.”**
- 4. “Then we use SPARQL to find the relationships connected to these seed nodes.”**
- 5. “We retrieve the source node, relationship, target node, and relationship metadata.”**
- 6. “After that, we perform graph traversal up to two hops.”**
- 7. “The first hop gets the directly connected nodes, and the second hop gets the next-level connected nodes.”**
- 8. “The retrieved nodes and relationships form the relevant subgraph for the query.”**

9. **“This retrieved subgraph is then passed to Cell 17 for ranking and context creation.”**

One simple paragraph to say

“In Cell 16, we perform the actual retrieval from our RDF Knowledge Graph. We first map the query entities to KG nodes and use them as seed nodes. Then we use SPARQL to retrieve their connected relationships and perform a two-hop graph traversal. The retrieved nodes and relationships give us the relevant part of the Knowledge Graph, which is passed to the next cell for ranking and answer generation.”

2. Cell 16 — What should I say?

You can say:

“Cell 16 is the main retrieval stage. First, we take the entities from the user query and map them to nodes in our RDF Knowledge Graph. These nodes become the seed nodes. Then we use SPARQL to find the relationships connected to these seed nodes. After that, we perform graph traversal up to two hops to retrieve additional connected nodes and relationships. The result is the relevant subgraph from our Knowledge Graph.”

Simple flow

```
graph TD; A[User Query] --> B[Query Entities]; B --> C[KG Node Matching]; C --> D[Seed Nodes]; D --> E[SPARQL]; E --> F[Connected Relationships]; F --> G[2-Hop Traversal]; G --> H[Relevant KG Subgraph];
```

User Query
↓
Query Entities
↓
KG Node Matching
↓
Seed Nodes
↓
SPARQL
↓
Connected Relationships
↓
2-Hop Traversal
↓
Relevant KG Subgraph



If they ask what SPARQL is doing

Say:

“SPARQL is the graph query language I use to query our RDF Knowledge Graph. It finds the source, predicate, target and relationship metadata connected to the seed nodes.”



3. Cell 17 — What should I say?

Say:

"Cell 17 takes the subgraph retrieved in Cell 16 and prepares the relevant information for the LLM. First, we rank the retrieved relationships based on their relevance, hop distance and relationship strength. Then we select the important relationships and retrieve the associated document content using SPARQL. Finally, we combine the relationships and document content into a structured context and pass it to the LLM for answer generation."

Simple flow

```
Cell 16
Retrieved Subgraph
↓
Rank Relationships
↓
Select Relevant Relationships
↓
Find Relevant Documents
↓
SPARQL → Document Content
↓
Build Context
↓
LLM
↓
Final Answer
```



4. If they ask: "How is this related to the paper?"

Say:

"I am referring to the SG-RAG paper for the overall subgraph-retrieval and graph-to-text approach. The paper retrieves a relevant subgraph from the Knowledge Graph, transforms the graph information into text, and provides it to the LLM for answer generation. Since our Knowledge Graph is RDF, I

Ask questions

Get This



5. One-minute complete explanation

If you want to explain Cell 16 + Cell 17 continuously, use this:

"For the retrieval part, I am using an RDF Knowledge Graph-based multi-hop SPARQL subgraph retriever. In Cell 16, the entities from the user query are mapped to nodes in our Knowledge Graph, and these become the seed nodes. We then use SPARQL to find the relationships connected to these nodes and perform a two-hop graph traversal to retrieve the relevant subgraph. In Cell 17, we process this retrieved subgraph by ranking the relationships, selecting the relevant documents, and fetching their content using SPARQL. We then build a structured context containing the graph relationships and document information and pass that context to the LLM to generate the final answer."

Yes, correct.

In your implementation, the **graph traversal** is performed through **SPARQL queries** over your RDF Knowledge Graph.

More precisely:

- **SPARQL** → finds the connected RDF relationships for the current seed/frontier nodes.
- **Your Python `traverse()` loop** → repeats that SPARQL query for each hop.
- **`MAX_HOPS = 2`** → limits the traversal to two hops.

So:

```
graph TD; A[Seed Node] --> B[SPARQL query → Hop 1]; B --> C[New connected nodes]; C --> D[SPARQL query → Hop 2]; D --> E[More connected nodes]; E --> F[Retrieved subgraph];
```

The important wording is:

“We implement multi-hop graph traversal by iteratively executing SPARQL queries over the RDF Knowledge Graph.”

Yes. For the **overall retriever**, you can explain it in these points:

Our Graph RAG Retriever

1. Query Entity Extraction

- We first identify the important entities from the user query.
- This is done in **Cell 15 using the LLM**.

2. Entity → KG Node Mapping

- The extracted entities are matched with the labels/nodes in our RDF Knowledge Graph.
- These matched nodes become the **seed nodes**.

3. SPARQL Graph Query

- We use **SPARQL** because our **Knowledge Graph is stored as RDF**.
- SPARQL finds the relationships connected to the seed nodes:
Source → Predicate → Target.

4. Multi-Hop Graph Traversal

- We expand the graph from the seed nodes.
- We currently use:
`MAX_HOPS = 2`

- Hop 1 → directly connected nodes.
- Hop 2 → next-level connected nodes.

5. Relevant Subgraph Retrieval

- The nodes and relationships collected through the traversal form the **query-specific relevant subgraph**.

6. Relationship Ranking

- In Cell 17, we rank the retrieved relationships based on:
 - seed/query match
 - hop distance
 - relationship strength
 - graph connectivity/degree

7. Document Retrieval

- From the selected KG relationships, we identify relevant document nodes.
- We then use **SPARQL** to retrieve their stored document content.

8. Context Construction

- We combine:
 - KG relationships
 - relationship descriptions
 - relevant document content
- This becomes the context for the LLM.

9. Answer Generation

- Finally, the LLM receives the **original question + retrieved KG context** and generates the grounded answer.

The complete flow

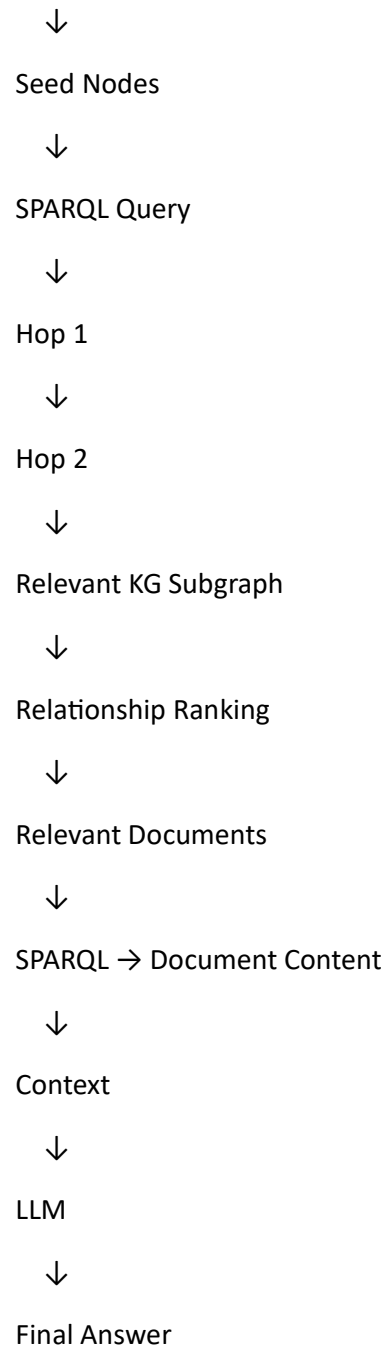
User Query



Entity Extraction



Entity → KG Node



One good paragraph for your meeting

“For our Graph RAG retriever, we first extract the important entities from the query and map them to nodes in our RDF Knowledge Graph. Since our KG is RDF, we use SPARQL to query the relationships connected to these seed nodes. We then perform a multi-hop traversal up to two hops to retrieve the relevant subgraph. After retrieval, we rank the relationships based on their relevance, hop distance, and relationship strength, retrieve the associated document content using SPARQL, and build the context that is finally passed to the LLM for answer generation.”

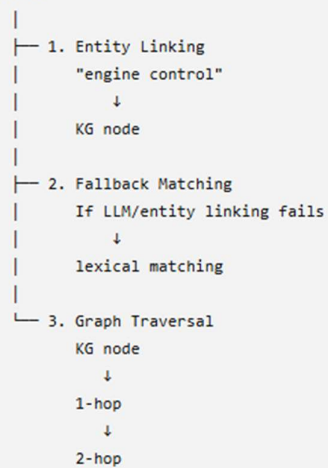
And if they ask “What is the retriever called?”:

“It is an RDF Knowledge Graph-based multi-hop SPARQL subgraph retriever.”

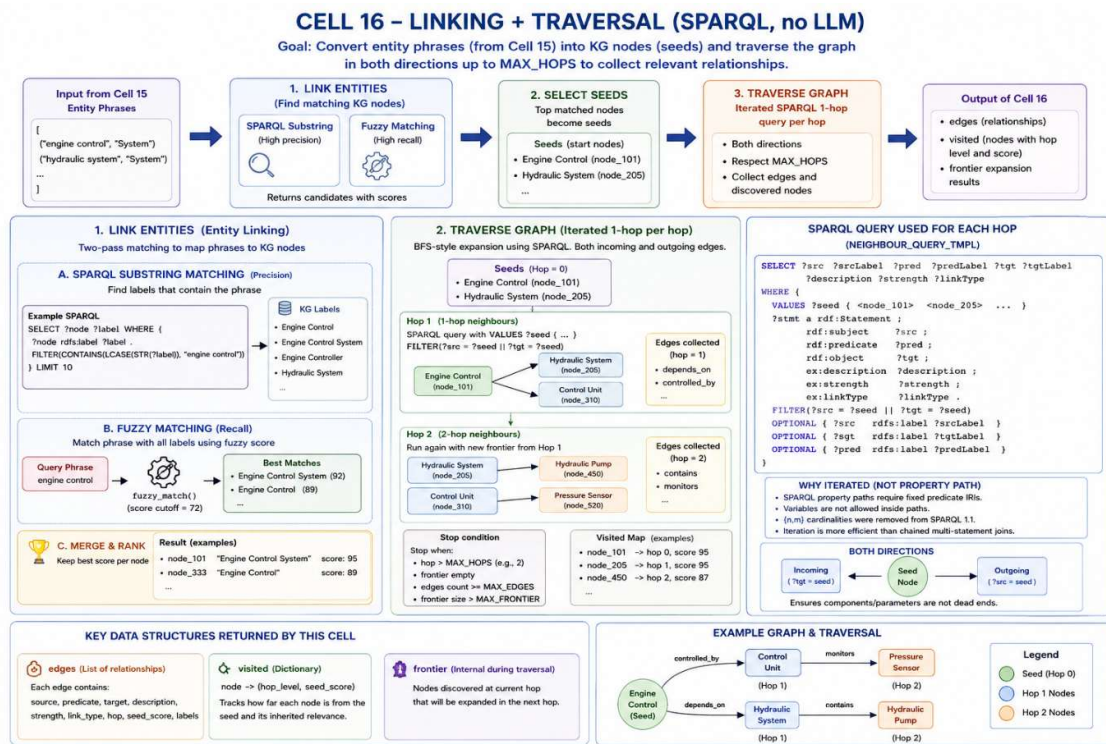
1. What is the logic of Cell 16?

There are actually three major jobs in this cell:

Cell 16



So:



Yes. **Cell 17 is the final stage of your Retriever pipeline.**

This is where the graph evidence found in **Cell 16** is:

- ranked** — decide which relationships are most relevant,
- ordered** — decide the sequence in which to present them,
- documents selected** — find the most relevant source documents,
- context assembled** — combine relationships + document text,
- and finally, **the LLM generates the answer** when kg_answer() is called.

2. One example

Let's use the same example we've been using.

Suppose the user asks:

"What does Engine Control depend on?"

Your KG contains:

Engine Control

|

| depends_on



Hydraulic System

|

| contains



Hydraulic Pump

Cell 16 might retrieve:

Edge 1:

Engine Control --depends_on--> Hydraulic System

hop = 1

strength = 10

seed_score = 95

Edge 2:

Hydraulic System --contains--> Hydraulic Pump

hop = 2

strength = 7

seed_score = 95

Edge 3:

Hydraulic Pump --monitors--> Pressure Sensor

hop = 2

strength = 4

seed_score = 95

Now Cell 17 has to decide:

Which of these relationships are most important?

That's what rank_edges() does.

3. rank_edges() — Decide what is important

Your code calculates:

score =

match quality

× hop decay

× relationship strength

× degree factor

Conceptually:

Seed matching

×

Distance from seed

×

Relationship strength

×

Graph connectivity

↓

Final relevance score

This is the **ranking logic** of your GraphRAG retriever.

4. What is seed_score?

Remember Cell 16?

It found:

User phrase:

"engine control"

↓

KG node:

Engine Control



match score = 95

That 95 becomes:

`e["seed_score"]`

So the retriever remembers:

"How confidently did we identify the starting node?"

A stronger entity match means a higher retrieval score.

5. What is HOP_DECAY doing?

You defined in Cell 14:

`HOP_DECAY = 0.45`

So:

Direct relationship

Engine Control



Hydraulic System

`hop = 1`

gets:

`HOP_DECAY0 = 1`

2-hop relationship

Engine Control



Hydraulic System



Hydraulic Pump

gets:

$HOP_DECAY^1 = 0.45$

So the farther away the information is, the less important it becomes.

This makes sense because:

Direct evidence is generally more relevant than distant graph evidence.

6. What is strength?

Your RDF relationship contains:

ex:strength

For example:

Engine Control

|
| depends_on
| strength = 10



Hydraulic System

Another relationship might have:

strength = 4

The stronger relationship gets a higher score.

So your retriever considers not only:

"Are these nodes connected?"

but also:

"How strong is this relationship?"

That's valuable in your GraphRAG design.

7. What is degree?

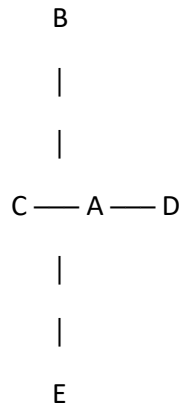
This is the NetworkX part.

`deg = _degree(e["source"]) + _degree(e["target"])`

Degree means approximately:

How many connections does a node have in the graph?

For example:



Node A has a high degree because it is connected to many nodes.

Your code uses degree as a small additional factor.

Why?

A highly connected node can represent an important central concept in the graph.

But notice:

Degree is only a small boost.

It doesn't dominate the score.

8. So what does the final score mean?

Suppose we have:

Edge 1

Engine Control → Hydraulic System

seed score = 95

hop = 1

strength = 10

This will get a high score.

Edge 2

Hydraulic System → Hydraulic Pump

seed score = 95

hop = 2

strength = 7

This gets a lower score because:

- it is farther away,
- and relationship strength is lower.

So:

Edge 1

★★★★★

Edge 2

★★★

This is how the retriever decides **which relationships matter more**.

9. Very important: Ranking vs Ordering

Your Cell 17 has **two different concepts**:

Ranking

Which relationships should we keep?

Ordering

In what order should we show the relationships to the LLM?

These are not the same.

Your comment specifically says this.

10. rank_edges() = WHICH evidence?

Suppose Cell 16 found:

A → B

$B \rightarrow C$

$C \rightarrow D$

$D \rightarrow E$

There may be 100 relationships.

`rank_edges()` scores them and says:

Keep:

$A \rightarrow B$

$B \rightarrow C$

$C \rightarrow D$

...

and removes the lower-ranked ones when you later apply:

`top = ranked[:MAX_TRIPLES]`

Since:

`MAX_TRIPLES = 40`

you keep at most 40.

11. `order_bfs()` = WHAT SEQUENCE?

Now suppose the selected relationships are:

$A \rightarrow B$

$B \rightarrow C$

$C \rightarrow D$

Instead of giving the LLM something random like:

$C \rightarrow D$

$A \rightarrow B$

$B \rightarrow C$

you arrange them in graph traversal order:

$A \rightarrow B$

$B \rightarrow C$

$C \rightarrow D$

This makes the context easier to follow.

So:

Ranking



Select important evidence

BFS ordering



Arrange selected evidence logically

Important:

BFS does NOT decide which edges are relevant.

rank_edges() decides that.

BFS only decides their presentation order.

12. Why BFS ordering?

Imagine explaining this to a human.

This is easier to understand:

Engine Control



Hydraulic System



Hydraulic Pump

than:

Hydraulic Pump → Pressure Sensor

Engine Control → Hydraulic System

Hydraulic System → Hydraulic Pump

The first one follows the graph naturally.

So the context has a logical flow:

Seed



Related entity



Next related entity



Next related entity

This is particularly useful when the LLM has to reason over connected relationships.

13. `pick_documents()` — Select source documents

Now you have relationships.

But relationships alone aren't enough.

For example:

Engine Control



depends_on



Hydraulic System

The LLM may also need the **actual source document content**.

So `pick_documents()` asks:

Which graph nodes involved in these high-ranking relationships actually contain document content?

Remember Cell 14 created:

`CONTENT_NODES`

which tells us which nodes have:

ex:documentContent

Then it scores those documents based on the relevance of the edges they're involved in.

Finally:

[[:MAX_DOCS]

keeps at most:

MAX_DOCS = 4

documents.

14. Why do we need documents if we already have graph relationships?

This is an important GraphRAG concept.

The graph gives you:

STRUCTURE

For example:

A → depends_on → B

But the document gives you:

DETAILED INFORMATION

For example:

Engine Control depends on the hydraulic system because the hydraulic actuator provides...

So your retriever combines:

Knowledge Graph

+

Original Document Content

This is what makes your retrieval richer.

15. fetch_documents()

Now the selected document nodes are sent to SPARQL.

It retrieves:

doc

label

content

Importantly, it does this in **one SPARQL query** for all selected documents.

Instead of:

Query document 1

Query document 2

Query document 3

Query document 4

it does conceptually:

SPARQL

↓

Give me content for:

Doc1

Doc2

Doc3

Doc4

That's more efficient.

16. build_context() — Very important

This is where everything comes together.

It creates:

RELATIONSHIPS

Engine Control --[depends_on, 10]--> Hydraulic System : ...

Hydraulic System --[contains, 7]--> Hydraulic Pump : ...

DOCUMENTS

--- Engine Control ---

[document text...]

--- Hydraulic System ---

[document text...]

So this function converts your retrieved graph + documents into a **single context string** for the LLM.

17. Why have both RELATIONSHIPS and DOCUMENTS?

Because they provide different information.

Relationships

Tell the LLM:

How are the entities connected?

Engine Control

↓ depends_on

Hydraulic System

Documents

Tell the LLM:

What does the source material actually say?

[full document text]

Together:

Graph structure

+

Source evidence

↓

Better grounded context

18. Context limits

You also control how much information goes to the LLM:

MAX_TRIPLES = 40

MAX_DOCS = 4

MAX_DOC_CHARS = 2500

MAX_CONTEXT_CHARS = 14000

This prevents:

1000 relationships

+

100 documents

+

huge XML contents

from being sent to the LLM.

Instead:

Top relationships

+

Top documents

↓

Controlled context

This is important for both **performance and relevance**.

19. TRIPLE_BUDGET_FRAC = 0.55

This means you reserve roughly:

55%

of the context budget for relationships.

So conceptually:

14,000 character context

|

|— ~55% → Relationships

|

|— remaining → Documents

This ensures your graph structure doesn't get completely crowded out by document text.

20. `kg_retrieve()` — Complete Retriever

This is the most important function in Cell 17.

It connects **Cells 15, 16 and 17 together**.

Look at the sequence:

```
phrases = extract_query_entities(question)
```

Cell 15

Question

↓

Entity phrases

Then:

```
seeds = link_entities(phrases)
```

Cell 16

Entity phrases

↓

KG nodes

↓

Seeds

Then:

```
edges, _ = traverse(seeds)
```

Cell 16

Seeds

↓

Graph traversal

↓

Edges

Then:

```
ranked = rank_edges(edges)
```

Cell 17

Edges

↓

Ranking

Then:

```
presented = order_bfs(top, seeds)
```

Selected edges

↓

BFS ordering

Then:

```
docs = fetch_documents(pick_documents(ranked))
```

Relevant edges

↓

Relevant documents

Then:

```
context, n_chars = build_context(presented, docs)
```

Relationships

+

Documents

↓

Final context

That is your **complete retrieval pipeline**.

21. kg_retrieve() is retrieval only

This comment is very important:

```
""""Retrieval only, no generation""""
```

That means:

`kg_retrieve()`

does **not ask the LLM to answer**.

It returns:

Question

Entity phrases

Seeds

Edges

Documents

Context

This is excellent for debugging your retriever.

You can inspect:

Did I find the right entity?

Did I choose the right seed?

Did I traverse the right relationships?

Did I retrieve the right documents?

before involving the answer-generating LLM.

22. `kg_answer()` — Generation

Finally:

`kg_answer(question)`

does:

Question

↓

`kg_retrieve()`

↓

Retrieved Context

↓

LLM



Answer

The LLM receives:

RELATIONSHIPS

+

DOCUMENTS

+

QUESTION

and is instructed:

Use only this context. Don't use outside knowledge. Don't guess.

This is what makes the final answer **grounded in your KG and source documents**.

23. Complete example

Let's walk through the entire Cell 17 using:

"What does Engine Control depend on?"

Cell 16 gives:

Seed:

Engine Control

Edges:

1. Engine Control

--depends_on-->

Hydraulic System

strength = 10

hop = 1

2. Hydraulic System

--contains-->

Hydraulic Pump

strength = 7

hop = 2

3. Hydraulic Pump

--monitors-->

Pressure Sensor

strength = 4

hop = 2

Step 1 — Rank

The scoring function considers:

Seed match

×

Hop distance

×

Relationship strength

×

Degree

Result might conceptually be:

1. Engine Control → Hydraulic System HIGH
2. Hydraulic System → Hydraulic Pump MEDIUM
3. Hydraulic Pump → Pressure Sensor LOW

Step 2 — Keep top relationships

MAX_TRIPLES = 40

So only the highest-quality evidence is retained.

Step 3 — BFS ordering

Instead of:

Hydraulic Pump → Pressure Sensor

Engine Control → Hydraulic System

Hydraulic System → Hydraulic Pump

it presents:

Engine Control → Hydraulic System

Hydraulic System → Hydraulic Pump

Hydraulic Pump → Pressure Sensor

Step 4 — Select documents

Suppose:

Engine Control → document available

Hydraulic System → document available

Hydraulic Pump → document available

The most relevant documents are selected.

Step 5 — Build context

The final context becomes approximately:

RELATIONSHIPS

Engine Control --[depends_on, 10]--> Hydraulic System

Hydraulic System --[contains, 7]--> Hydraulic Pump

DOCUMENTS

--- Engine Control ---

[actual source document text]

--- Hydraulic System ---

[actual source document text]

Step 6 — LLM

Now the LLM receives:

Question:

What does Engine Control depend on?

Context:

Engine Control --depends_on--> Hydraulic System

...

It can answer:

Engine Control depends on the Hydraulic System.

And it can provide the document source.

24. What is the actual role of Cell 17?

This is the key point.

Cell 17 is:

The ranking, filtering, document selection, context assembly, and answer-generation stage of your GraphRAG pipeline.

Cell 16 says:

"Here are the relationships I found."

Cell 17 says:

"Which of these relationships are most relevant, which source documents should I use, how should I organize the evidence, and what context should I give to the LLM?"

25. Is Cell 17 necessary?

For your GraphRAG retriever: YES, strongly.

Without ranking:

Cell 16

↓

Maybe hundreds of edges

↓

LLM

Lots of noise.

With Cell 17:

Cell 16

↓

Hundreds of edges

↓

Ranking

↓

Top relevant edges

↓

Relevant documents

↓

Controlled context

↓

LLM

That's much better.

26. One important distinction across your cells

You can now explain your retriever as **four stages**:

Cell **Main responsibility** **Simple meaning**

Cell 14 Preparation "Prepare the graph for retrieval."

Cell 15 Query understanding "What entities is the user asking about?"

Cell 16 Graph retrieval "Find those entities and walk through their relationships."

Cell 17 Ranking + context "Choose the best evidence and prepare it for the LLM."

So the entire logic is:

USER QUESTION

|



┌──────────────────┐

| CELL 15 |

| Entity Extraction |

| LLM |

└──────────┴────────┘

|



Entity Phrases

|



┌──────────────────┐

| CELL 16 |

| Entity Linking |

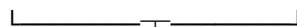
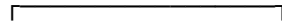
| + |

| Graph Traversal |

└──────────┴────────┘

|





Retrieved Context



LLM

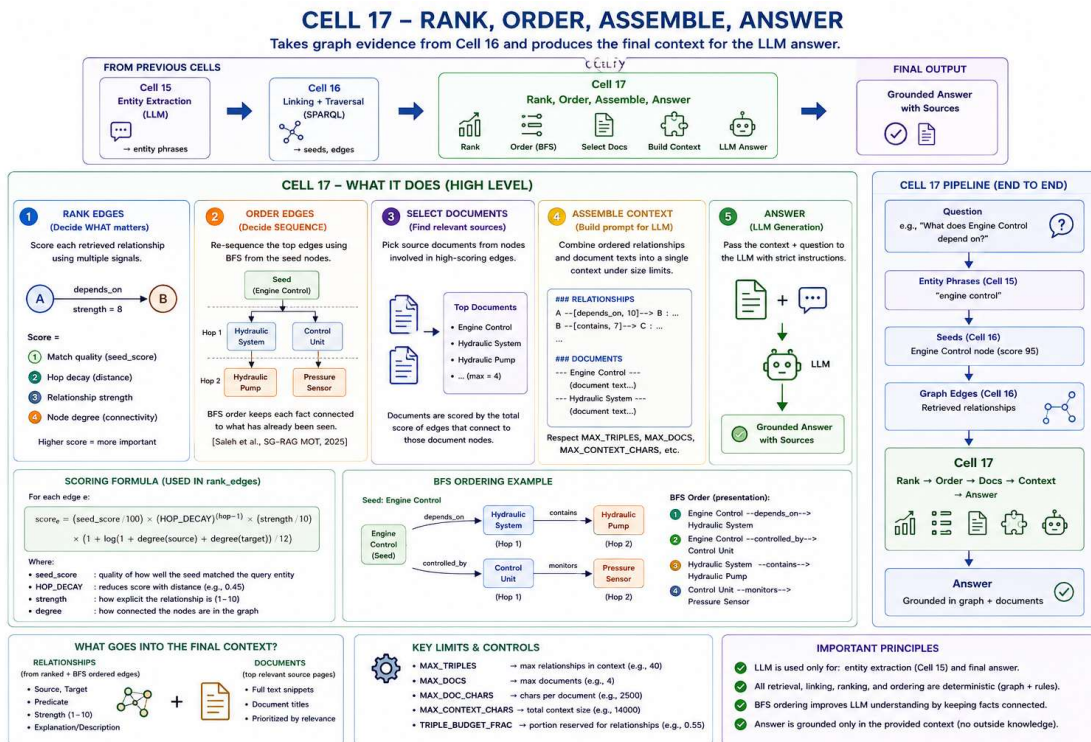


ANSWER

The simplest explanation of Cell 17

"Cell 17 takes the graph relationships retrieved in Cell 16, scores them using query-match quality, graph distance, relationship strength, and node connectivity, keeps the most relevant evidence, orders it using graph traversal order, retrieves the corresponding source documents, and combines everything into a controlled context that is finally passed to the LLM for a grounded answer."

One especially important point for your POC: **Cell 17 is where your GraphRAG becomes a ranked/hybrid retrieval system rather than simply a graph traversal system.**



es. For your meeting, I would explain the **Retriever** part as a **simple 4-stage flow**, corresponding to Cells 14–17.

GraphRAG Retriever — What to explain in the meeting

1. Retriever Setup — Cell 14

- I first **prepare and validate the RDF Knowledge Graph** for retrieval.
- I create an index between **KG node labels and their RDF nodes** so that user terms can be mapped to graph entities.
- I identify which nodes contain the **actual document content** for grounded retrieval.
- I verify that the graph contains the required **relationships/reified statements** for traversal.
- I configure retrieval parameters such as **maximum hops, hop decay, number of seeds, and context limits**.

In simple words:

"Cell 14 prepares the Knowledge Graph and makes sure it is ready for retrieval."

2. Query Entity Extraction — Cell 15

- The user gives a **natural-language question**.
- I use the LLM only to extract the **important entity phrases** from the question.
- For example:

Question:

"What does the Engine Control depend on?"

↓

Entity:

"Engine Control"

- The LLM is **not retrieving the answer here**.
- It is only helping convert the natural-language query into something that can be searched in the KG.

In simple words:

"Cell 15 understands what entities the user is asking about."

3. Entity Linking + Graph Traversal — Cell 16

This is the **main graph retrieval stage**.

- The extracted entity phrase is matched against KG labels.
- I use **SPARQL substring matching** for precise matches.
- I also use **fuzzy matching** to handle variations in the user's wording.
- The best matching KG nodes become **seed nodes**.
- From those seed nodes, I perform graph traversal using **SPARQL queries**.
- MAX_HOPS = 2 means I can explore up to **two hops from the seed**.
- The traversal works in **both directions**, so incoming and outgoing relationships can be found.
- I collect relationship information such as:
 - source
 - predicate

- target
- relationship description
- relationship strength
- hop distance

Important distinction to say:

"MAX_HOPS controls how far I traverse, while SPARQL is used to query the RDF graph and retrieve the relationships at each hop."

In simple words:

"Cell 16 finds the correct graph node and walks through its connected nodes to collect relevant graph evidence."

4. Ranking + Context Assembly — Cell 17

After traversal, I may have many relationships, so I need to identify the most relevant ones.

I rank relationships using:

Seed Match Quality

×

Hop Decay

×

Relationship Strength

×

Node Degree

- **Seed score** → how well the query matched the starting node.
- **Hop decay** → direct relationships are preferred over distant relationships.
- **Relationship strength** → stronger relationships get higher priority.
- **Degree** → gives a small boost to highly connected nodes.

Then:

- Keep the highest-ranked relationships.
- Use **BFS ordering** to present the selected relationships in graph-traversal sequence.
- Select the most relevant source documents.

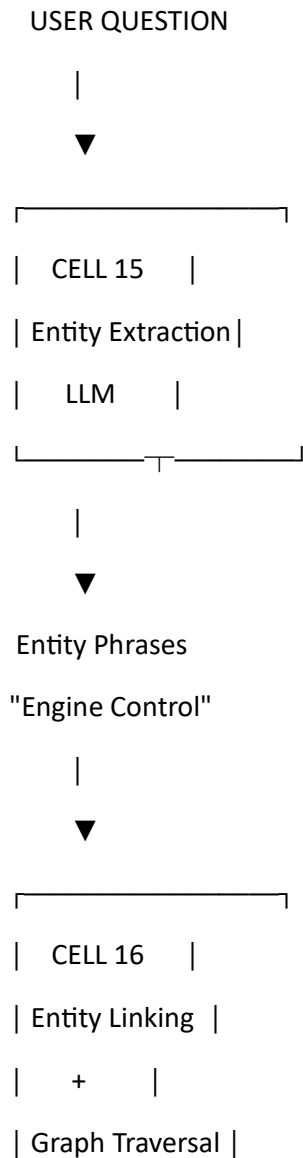
- Retrieve their actual document content using **SPARQL**.
- Combine **graph relationships + document content** into the final context.
- Send this context to the LLM for the final grounded answer.

In simple words:

"Cell 17 filters and organizes the retrieved graph evidence and combines it with the original document content before sending it to the LLM."

The complete Retriever flow

You can show this in the meeting:



| SPARQL |

└──┴──┘

|



Graph Relationships

|



└──────────┘

| CELL 17 |

| Ranking |

| BFS Ordering |

| Document Select |

| Context Build |

└──┴──┘

|



Retrieved Context

└──┬──┘

| |

KG Relationships Documents

| |

└──┴──┘



LLM

|



GROUNDED ANSWER



If they ask "What exactly did you use?"

You can answer:

"I implemented a knowledge-graph-based retriever over the RDF graph. The pipeline uses LLM-based entity extraction, SPARQL and fuzzy matching for entity linking, iterative SPARQL-based multi-hop graph traversal, bidirectional relationship exploration, relevance scoring using seed similarity, hop decay and relationship strength, BFS-based context ordering, and finally combines graph evidence with source document content before passing the context to the LLM."

If they ask "Why not just use vector search?"

Say:

"Vector search is good for semantic similarity, but it doesn't explicitly capture the relationships between entities. My graph retriever first identifies the relevant entity and then follows the actual relationships stored in the RDF graph, allowing it to retrieve multi-hop and relational context."

One important point for your meeting

Don't say:

"The LLM does the retrieval."

Instead say:

"The LLM is used for query entity extraction and final answer generation. The actual retrieval is primarily deterministic and graph-based, using SPARQL, entity matching, graph traversal, ranking, and document retrieval."

That is probably the **most important architectural point** to communicate.

30-second version

If you have very little time:

"Our retriever takes the user's question, extracts the important entities using an LLM, links those entities to nodes in the RDF Knowledge Graph using SPARQL and fuzzy matching, and uses those nodes as seeds for multi-hop graph traversal. The traversal is performed iteratively using SPARQL, up to two hops, in both directions. The retrieved relationships are then ranked using query-match quality, hop distance, relationship strength and graph connectivity. Finally, we select the most relevant source documents, combine their content

with the graph relationships, and pass this grounded context to the LLM to generate the answer."